

Rapport — Libreville Eats (Groupe 3)

[Rapport final — Projet NoSQL : Libreville Eats](#)

[Table des matières](#)

[1. Introduction](#)

[2. Contexte métier](#)

[3. Modélisation NoSQL](#)

[4. Implémentation](#)

[5. Requêtes et agrégations](#)

[6. Indexation](#)

[7. Analyse des performances](#)

[8. Comparaison SQL contre NoSQL](#)

[9. Difficultés rencontrées](#)

[10. Conclusion](#)

Rapport final — Projet NoSQL : Libreville Eats

Module : Bases de données NoSQL — MongoDB — Master I **Année académique :** 2025-2026

Groupe : Groupe 3 — Thème « Application de Livraison de Repas » **Membres :**

- BOUKINDA MAMBOUNDOU Jude
- MAVOUNGOU Serge Murlain
- BENOME NTOUTOUME Élise Josita

Dépôt GitHub : `projet-nosql-groupeN3` **Focus NoSQL imposé par le sujet :** données temps réel et caching (complémentarité MongoDB / Redis), modélisation d'un menu qui change fréquemment.

Table des matières

1. Introduction
 2. Contexte métier
 3. Modélisation NoSQL
 4. Implémentation
 5. Requêtes et agrégations
 6. Indexation
 7. Analyse des performances
 8. Comparaison SQL contre NoSQL
 9. Difficultés rencontrées
 10. Conclusion
-

1. Introduction

Libreville Eats est une application de livraison de repas à domicile pour la ville de Libreville (Gabon), développée dans le cadre du projet de fin de module *Bases de données NoSQL*. Le sujet attribué au Groupe 3 impose de modéliser un système où trois types d'acteurs interagissent autour d'une commande : un **client** qui parcourt les restaurants partenaires et commande, un **restaurateur** qui gère un menu changeant (plats du jour, ruptures de stock), et un **livreur** dont la position GPS doit être suivie en temps réel jusqu'à la livraison.

Le socle obligatoire du cahier des charges demande : un modèle de données documentaire justifié, une base peuplée de données réalistes, les opérations CRUD de base, des requêtes avancées et des pipelines d'agrégation, une stratégie d'indexation mesurée par `explain('executionStats')`, et le présent rapport. Au-delà de ce socle, notre groupe a choisi de réaliser également l'intégralité des livrables bonus du cahier des charges : une API REST complète (Node.js/Express/Mongoose/JWT), une stratégie de cache Redis documentée, et un frontend Next.js de démonstration permettant de manipuler l'application comme un utilisateur réel plutôt que de se limiter à des scripts isolés exécutés dans `mongosh`.

Ce choix a une conséquence directe sur la lecture de ce rapport : chaque concept NoSQL étudié (embedding/referencing, agrégation, indexation) est systématiquement illustré à deux niveaux — d’abord par un script `mongosh` autonome et minimal (dossiers `scripts/`, `data/`, `explain/`, conformes à la structure de dépôt imposée), puis par son usage réel dans le code de production (`src/`), ce qui permet de vérifier que les choix de modélisation tiennent la route au-delà de l’exercice académique.

2. Contexte métier

Le marché de la livraison de repas à domicile à Libreville présente des contraintes métier spécifiques, qui ont directement orienté nos choix de modélisation :

- **Connectivité mobile variable.** Une large partie des utilisateurs accède au service depuis un réseau mobile dont la latence et la disponibilité varient fortement. L’API doit rester réactive même en cas de pic de charge sur MongoDB — d’où le recours à Redis comme couche de cache devant la base de persistance (détaillé en section 4.3).
- **Suivi en temps réel.** Un client doit pouvoir visualiser, en direct, la progression de sa commande et la position de son livreur jusqu’à la livraison. Cette exigence impose une structure de données capable d’accumuler des relevés horodatés fréquents (toutes les 20 à 30 secondes) sans dégrader les performances de lecture sur le document parent.
- **Menus changeants.** Contrairement à un catalogue e-commerce statique, un menu de restaurant évolue plusieurs fois par jour : un plat épuisé doit disparaître immédiatement des résultats de recherche, sans pour autant invalider les commandes déjà passées qui le référencent.
- **Diversité des rôles avec un compte unique.** Un même individu peut, dans la vraie vie, être client d’un restaurant et restaurateur d’un autre établissement. Le système retient donc une collection `users` unique avec un champ `role` (`CUSTOMER`, `VENDOR`, `DELIVERER`, `ADMIN`), plutôt que des collections cloisonnées par type d’acteur — ce qui simplifie l’authentification (un seul mécanisme JWT pour tous les rôles) au prix d’une discipline de validation par rôle au niveau applicatif (middleware `authorize(...roles)`).
- **Ancrage local.** Le cahier des charges impose des données réalistes et ancrées dans le contexte gabonais. Les noms de personnes, les quartiers de Libreville (Nombakélé, Batavéa, Akébé, Glass, Owendo, Akanda...), les plats (Poulet Nyembwe, Odika, Brochettes...), les numéros de téléphone au format `+241` et les montants en FCFA structurent l’intégralité du jeu de données et des exemples de ce rapport.

Ces contraintes ont structuré l’ensemble des arbitrages de modélisation détaillés dans la section suivante : ce qui doit être lu instantanément avec son parent (embedding) versus ce qui a un cycle de vie propre et une cardinalité non bornée (referencing).

3. Modélisation NoSQL

3.1 Vue d’ensemble des collections

Le modèle s’appuie sur quatre collections MongoDB principales :

Collection	Rôle	Documents minimum (cahier des charges)
users	Comptes (clients, restaurateurs, livreurs, admin)	30-50
restaurants	Établissements partenaires, avec menus embarqués	30-50
commandes	Commandes passées, snapshot des articles, historique de statuts, tracé GPS	30-50
deliverers	Profils métier des livreurs (véhicule, position, performance)	30-50

Le schéma détaillé de chaque collection (champs, types, exemples JSON) est fourni dans `conception/modele-donnees.md / .pdf`, et représenté visuellement dans `conception/schema.png`. Nous en reprenons ici la synthèse et la justification, condition nécessaire à la compréhension du reste du rapport.

3.2 Schéma `users`

```
{
  "_id": ObjectId,
  "email": "paul.ogandaga@gmail.com",
  "password": "$2b$10$...",
  "role": "CUSTOMER",
  "profile": { "firstName": "Paul", "lastName": "Ogandaga", "phone":
    "+24106777777" },
  "addresses": [
    { "label": "Domicile", "district": "Batavéa",
      "coordinates": { "type": "Point", "coordinates": [9.4720, 0.4280] },
      "isDefault": true }
  ],
  "favoriteRestaurants": [ObjectId, ObjectId],
  "isActive": true,
  "metadata": { "createdAt": ISODate, "updatedAt": ISODate }
}
```

Les adresses sont **embarquées** : un utilisateur en a rarement plus de 2 ou 3, elles sont toujours lues avec le profil (sélection de l'adresse de livraison), et leur cycle de vie est entièrement subordonné au compte utilisateur — il n'existe aucun cas d'usage où l'on chercherait une adresse indépendamment de son propriétaire.

3.3 Schéma `restaurants`

```
{
  "_id": ObjectId,
  "name": "Chez Albert - Libreville",
  "owner_id": ObjectId,
  "address": { "district": "Nombakélé",
    "coordinates": { "type": "Point", "coordinates": [9.4583, 0.4162] } },
  "isOpen": true,
  "rating": 4.5,
  "menus": [
    { "name": "Menu Principal",
      "dishes": [
        { "_id": ObjectId, "name": "Poulet Nyembwe", "price": 3500,
          "category": "Plats Principaux", "isAvailable": true, "quantity": 20 }
      ]
    },
    ...
  ],
  "deliveryZones": [{ "zone": "Nombakélé", "deliveryFee": 1000, "deliveryTime":
    30 }],
  "metadata": { "createdAt": ISODate, "totalOrders": 342, "totalRevenue": 1190000 }
}
```

Les **menus et leurs plats sont embarqués**. Trois raisons motivent ce choix : (1) ils sont systématiquement lus ensemble — afficher la page d'un restaurant signifie toujours afficher son menu complet, il n'existe aucune route qui lirait un plat indépendamment de son restaurant ; (2) la taille du sous-document reste bornée (un restaurant gabonais propose en pratique quelques dizaines de plats, très loin de la limite de 16 Mo d'un document MongoDB) ; (3) la mise à jour d'un statut de disponibilité (`isAvailable`) ou d'une quantité est une opération atomique sur un seul document via `$set` , ce qui correspond exactement au besoin métier « le restaurateur marque un plat en rupture en un clic ». Les **zones de livraison** suivent la même logique : taille fixe et petite, jamais interrogées hors du restaurant parent.

Le champ `owner_id` , en revanche, est **référéncé** : un restaurateur peut posséder plusieurs établissements, et le profil utilisateur (authentification, mot de passe) évolue indépendamment du restaurant — l'embarquer dupliquerait des données sensibles et volatiles dans chaque document restaurant.

3.4 Schéma commandes

```
{
  "_id": ObjectId,
  "customer_id": ObjectId,
  "restaurant_id": ObjectId,
  "deliverer_id": ObjectId,
  "items": [
    { "dishId": ObjectId, "dishName": "Poulet Nyembwe", "quantity": 2,
      "unitPrice": 3500, "totalPrice": 7000 }
  ],
  "pricing": { "subtotal": 8500, "deliveryFee": 1000, "total": 9500, "currency":
    "FCFA" },
  "status": "DELIVERY_IN_PROGRESS",
  "statusHistory": [
    { "status": "PENDING", "timestamp": ISODate, "note": "Commande créée" },
    { "status": "CONFIRMED", "timestamp": ISODate, "note": "Restaurant a
      confirmé" }
  ],
  "deliveryTracking": [
    { "timestamp": ISODate, "coordinates": { "type": "Point", "coordinates":
      [9.4583, 0.4162] }, "speed": 25.5 }
  ],
  "deliveryInfo": { "type": "DELIVERY", "recipientName": "Jean Dupont" },
  "payment": { "method": "CASH", "status": "PENDING" },
  "metadata": { "createdAt": ISODate, "updatedAt": ISODate }
}
```

`commandes` est la collection la plus riche en arbitrages :

- `items[]` **est embarqué en mode « snapshot »**, et non sous forme de référence vers `restaurants.menus.dishes`. C'est un choix volontairement différent de l'intuition naïve (« le plat existe déjà ailleurs, autant le référencer ») : si le restaurateur change le prix du Poulet Nyembwe demain, une commande passée hier doit conserver le prix payé au moment de l'achat. Référencer le plat romprait la facture historique dès le premier changement de menu. C'est l'exemple le plus net, dans tout le projet, d'un cas où le « bon » choix NoSQL contredit l'instinct de normalisation hérité du relationnel.
- `statusHistory[]` **est embarqué** : c'est un audit trail de taille prévisible (au plus 8 transitions, une par statut possible de la machine à états — voir section 4.2), systématiquement consulté avec la commande (jamais de requête « toutes les transitions vers CONFIRMED, tous clients confondus »).
- `deliveryTracking[]` **est embarqué**, mais avec une réserve documentée : ce tableau croît au rythme d'un point GPS toutes les 20-30 secondes pendant la phase de livraison. Pour une livraison de 30 minutes, cela représente 60 à 90 points — largement compatible avec la limite de 16 Mo, mais le choix a été explicitement réévalué en section 9 (difficultés rencontrées) et en section 4.3 (rôle de Redis), car c'est le point du modèle le plus sensible à un changement d'échelle.
- `customer_id`, `restaurant_id`, `deliverer_id` **sont référencés** : ce sont des entités à cardinalité illimitée (un client passe N commandes), à cycle de vie propre, et dont certains

attributs (note du restaurant, disponibilité du livreur) changent indépendamment de la commande — les embarquer dupliquerait des données rapidement obsolètes.

3.5 Schéma `deliverers`

```
{
  "_id": ObjectId,
  "user_id": ObjectId,
  "personalInfo": { "firstName": "Ahmed", "lastName": "Makanda", "phone":
    "+241061234567" },
  "vehicleInfo": { "type": "MOTORCYCLE", "licensePlate": "LA2024ABC" },
  "currentLocation": { "type": "Point", "coordinates": [9.4583, 0.4162],
    "lastUpdated": ISODate },
  "isActive": true,
  "isAvailable": true,
  "performanceMetrics": { "totalDeliveries": 342, "averageRating": 4.7,
    "averageDeliveryTime": 28 },
  "metadata": { "createdAt": ISODate }
}
```

`deliverers` est séparé de `users` (référence `user_id`) pour la même raison que `restaurants.owner_id` : isoler les données d'authentification (volatiles, sensibles) des données métier du livreur (véhicule, position courante, métriques de performance), qui évoluent à un rythme et avec des permissions de lecture très différents.

3.6 Tableau récapitulatif embedding / referencing

Donnée	Choix	Justification synthétique
Adresses d'un utilisateur	Embedding	Petites, toujours lues avec le profil
Menus & plats d'un restaurant	Embedding	Toujours lus ensemble, taille bornée, mise à jour atomique du statut
Zones de livraison d'un restaurant	Embedding	Taille fixe, pas de croissance indépendante
<code>items[]</code> d'une commande	Embedding (snapshot)	Le prix/nom au moment de l'achat ne doit pas suivre l'évolution du menu
<code>statusHistory[]</code> d'une commande	Embedding	Audit trail borné, toujours consulté avec la commande
<code>deliveryTracking[]</code> d'une commande	Embedding (avec vigilance)	Time-series local, mais croissance à surveiller (voir §9)
<code>customer_id</code> / <code>restaurant_id</code> / <code>deliverer_id</code>	Referencing	Cardinalité illimitée, cycle de vie propre, attributs qui évoluent indépendamment
<code>owner_id</code> d'un restaurant	Referencing	Un propriétaire peut posséder plusieurs restaurants

Donnée	Choix	Justification synthétique
<code>user_id</code> d'un livreur	Referencing	Sépare authentification (sensible) et données métier (véhicule, position)

Règle générale appliquée : nous embarquons une donnée quand elle est (a) lue quasi systématiquement avec son document parent, (b) de taille bornée et prévisible, (c) peu sujette à une croissance non corrélée au document parent. Nous la référençons quand l'entité a un cycle de vie propre, une cardinalité élevée ou illimitée, ou est partagée entre plusieurs documents parents.

4. Implémentation

4.1 Jeu de données et opérations CRUD

Le jeu de données réaliste est fourni à deux niveaux, conformément à la structure de dépôt attendue :

- `data/seed.js` : version `mongosh` pure (`insertMany()`), chargeable directement sans Node.js ni Mongoose — c'est le script de référence pour la correction et la soutenance.
- `scripts/seed/index.js` + `scripts/data/gabon-data.js` : générateur utilisé par l'application réelle (`npm run seed`), qui produit le même jeu de données via les modèles Mongoose (validation de schéma incluse).

Les deux scripts produisent un jeu de données identique en substance : **30 restaurants réels** de Libreville/Akanda/Owendo (parmi lesquels des enseignes connues comme KFC, PAUL, Blé Doré, La Braise, Bantu) avec leurs vrais plats, horaires et zones de livraison ; **90 comptes utilisateurs** (30 clients, 30 restaurateurs — un par restaurant — et 30 livreurs), tous avec des noms gabonais et un mot de passe de démonstration (`TestPass123` , volontairement en clair dans le seed pour faciliter la correction, jamais utilisé en environnement réel) ; **30 profils livreurs** ; et un échantillon de commandes cohérentes avec ces entités. Pour permettre une démonstration en direct des fonctionnalités de filtrage du sujet, **18 des 30 restaurants** sont volontairement marqués fermés (`isOpen: false`) et **10 % des plats** de chaque restaurant sont marqués en rupture de stock (`isAvailable: false`).

Les opérations CRUD du socle obligatoire sont regroupées dans `scripts/01-crud.js` :

Opération	Exemple dans le script
<code>insertOne</code>	Création d'un nouveau restaurant (Akébé Délices)
<code>insertMany</code>	Création de deux nouveaux clients
<code>find</code> (filtre + projection)	Restaurants ouverts, projection sur <code>name / address.district / rating</code>
<code>updateOne (\$set)</code>	Fermeture d'un restaurant et mise à jour de <code>metadata.updatedAt</code>
<code>updateMany (\$set)</code>	Initialisation de <code>emailVerified: false</code> pour les nouveaux clients

Opération	Exemple dans le script
<code>\$inc</code>	Incrément de <code>metadata.totalOrders</code> et <code>metadata.totalRevenue</code> d'un restaurant
<code>\$push</code>	Ajout d'une transition dans <code>statusHistory[]</code> d'une commande
<code>\$pull</code>	Retrait d'un restaurant de <code>favoriteRestaurants[]</code> d'un client
<code>deleteOne</code>	Suppression du restaurant de démonstration créé en début de script
<code>deleteMany</code>	Purge des deux clients de démonstration insérés plus haut

4.2 Machine à états des commandes

Le défi du sujet impose de suivre le statut d'une commande « via une machine à états (champ statut et horodatage des transitions) ». Cette logique est centralisée dans `src/utils/orderStateMachine.js`, qui définit huit statuts (`PENDING`, `CONFIRMED`, `PREPARING`, `READY_FOR_DELIVERY`, `DELIVERY_IN_PROGRESS`, `DELIVERED`, `CANCELLED`, `FAILED`) et une table de transitions valides :

```
const VALID_TRANSITIONS = {
  PENDING: ['CONFIRMED', 'CANCELLED', 'FAILED'],
  CONFIRMED: ['PREPARING', 'CANCELLED', 'FAILED'],
  PREPARING: ['READY_FOR_DELIVERY', 'CANCELLED', 'FAILED'],
  READY_FOR_DELIVERY: ['DELIVERY_IN_PROGRESS', 'CANCELLED'],
  DELIVERY_IN_PROGRESS: ['DELIVERED', 'FAILED'],
  DELIVERED: [], // état final
  CANCELLED: [], // état final
  FAILED: [], // état final
};
```

Aucun contrôleur n'écrit directement `commande.status` : chaque mise à jour passe par `isValidTransition(current, next)`, qui rejette toute transition non autorisée (par exemple, repasser une commande `DELIVERED` à `PREPARING`). Chaque transition validée est à la fois appliquée au champ `status` et ajoutée à `statusHistory[]` avec un horodatage, ce qui donne une traçabilité complète sans collection séparée.

4.3 Complémentarité MongoDB / Redis

Le focus NoSQL du sujet (« données temps réel et caching ») est traité dans `docs/design/PHASE1_REDIS_CACHING.md`. La note de conception y identifie cinq cas d'usage où MongoDB seul deviendrait un goulot d'étranglement, et propose pour chacun une structure Redis adaptée :

Donnée	Pourquoi pas seulement MongoDB	Structure Redis	TTL
Panier en cours		Hash <code>cart:{userId}</code>	24 h

Donnée	Pourquoi pas seulement MongoDB	Structure Redis	TTL
	Une écriture à chaque ajout/retrait d'article serait coûteuse en IO disque		
Statut de livraison « live »	Lecture fréquente par WebSocket, latence MongoDB (50-100 ms) trop élevée pour un suivi fluide	Hash order: <code>{orderId}:status</code>	2 h
Positions GPS du livreur	Un push toutes les 20-30 s dans <code>deliveryTracking[]</code> ferait croître le document à chaque commande	Redis Stream <code>deliverer:gps:</code> <code>{id}:stream</code> , capé à 500 entrées (XTRIM MAXLEN ~ 500)	Fenêtre glissante
Favoris	Lecture/écriture très fréquente, peu critique en durabilité immédiate	Set favorites: <code>{userId}</code>	7 jours
Disponibilité d'un plat	Champ <code>isAvailable</code> lu à chaque affichage de menu, modifié occasionnellement	Hash <code>dish:available:</code> <code>{restaurantId}:</code> <code>{dishId}</code> (pattern cache-aside)	5 min

Le principe retenu est celui du **cache-aside avec repli (fallback) sur MongoDB** : en cas d'indisponibilité de Redis, l'application continue de fonctionner en lisant directement MongoDB (latence dégradée mais service non interrompu) — c'est la stratégie effectivement implémentée dans `src/config/redis.js` et `src/services/cacheService.js` : le client Redis dégrade silencieusement en « cache désactivé » si la connexion échoue, et `cacheService` traite un client déconnecté comme un cache *miss* plutôt que de lever une exception. Cette approche répond directement à la contrainte de connectivité mobile variable identifiée en section 2 : la disponibilité du service ne dépend jamais de la disponibilité du cache.

4.4 Gestion des ruptures de stock

Le livrable « modélisation d'un menu avec gestion des ruptures de stock par plat » se traduit, dans `src/models/Restaurant.js`, par deux champs sur chaque plat embarqué : `isAvailable` (booléen, bascule manuelle du restaurateur) et `quantity` (entier, décrémenté à chaque commande). Un endpoint dédié (`restaurantController.js`, fonction `listOpenRestaurantsWithAvailability`) interroge ce double critère via `$elemMatch` imbriqué :

```
db.restaurants.find({
  isOpen: true,
  menus: { $elemMatch: { dishes: { $elemMatch: { isAvailable: true, quantity: {
    $gt: 0 } } } } }
});
```

Cette requête répond directement au livrable du sujet : « endpoint listant les restaurants ouverts avec au moins un plat disponible ».

4.5 Suivi GPS du livreur (défi technique)

Le défi technique du sujet — « modéliser le suivi GPS du livreur pendant la course (tableau de positions horodatées) » — est implémenté par le champ `deliveryTracking[]` décrit en section 3.4, alimenté en temps réel via Socket.io : le livreur émet un événement `location:update`, reçu côté serveur (`src/socket/index.js`), qui met à jour `deliverers.currentLocation` et rediffuse la position (`location:updated`) aux clients abonnés à la room `order:<id>`, avant d'être périodiquement persistée dans `commandes.deliveryTracking[]`. Ce découplage (Redis/Socket.io pour le flux temps réel à haute fréquence, MongoDB pour la persistance à fréquence réduite) est la matérialisation concrète de la note de conception Redis décrite en 4.3.

5. Requêtes et agrégations

5.1 Requêtes avancées

`scripts/02-requetes.js` couvre l'ensemble des opérateurs exigés par le socle obligatoire :

- **Comparaison** : `$gt` (commandes dont le total dépasse 5000 FCFA), `$lt` (restaurants notés en dessous de 4.3).
- **Logique sur tableaux/ensembles** : `$in` (commandes dans un sous-ensemble de statuts actifs).
- **Expressions régulières** : recherche insensible à la casse sur le nom d'un restaurant (`/gabon|albert/i`), et sur le domaine d'un email (`/@gmail\.com$/`).
- **Tri** : restaurants par note décroissante, commandes par date de création décroissante.
- **Pagination** : `skip/limit` sur les commandes triées par date (page 1, page 2).
- **Comptage** : `countDocuments` sur les restaurants ouverts et sur les commandes actives.

5.2 Pipelines d'agrégation

`scripts/03-agregations.js` contient trois pipelines combinant systématiquement `$match`, `$group`, `$sort` et `$lookup`, repris de la logique réellement exécutée en production (`scripts/aggregations/*.js`, via Mongoose, exposée par `npm run aggregate:all`) :

Pipeline 1 — statistiques de commandes par restaurant : `$group` calcule, par `restaurant_id`, le nombre de commandes, le chiffre d'affaires, le panier moyen et la répartition livrées/annulées ; `$lookup` joint la collection `restaurants` pour récupérer le nom et le quartier ; `$unwind` aplatit le tableau résultant du `$lookup` ; `$project` formate la sortie ; `$sort` classe par chiffre d'affaires décroissant.

Pipeline 2 — performance des livreurs : `$match` filtre les commandes ayant un livreur assigné ; `$group` calcule le nombre de livraisons réussies, le chiffre d'affaires généré et le taux d'annulation par `deliverer_id` ; `$lookup + $unwind` récupèrent le nom et le type de véhicule du livreur ; `$sort` classe par nombre de livraisons.

Pipeline 3 — analyse des revenus : deux variantes, par moyen de paiement (`$match` sur le statut de paiement, `$group` par `payment.method`) et par restaurant (`$group` par `restaurant_id` , puis `$lookup` vers `restaurants`), toutes deux triées par revenu décroissant.

Ces trois pipelines répondent directement aux exigences du sujet : « requête d’agrégation des recettes par événement » s’y transpose comme « recettes par restaurant », et la logique de jointure (`$lookup`) est systématiquement utilisée pour enrichir un résultat agrégé d’identifiants avec les informations lisibles (nom du restaurant, nom du livreur) — exactement le rôle que jouerait un `JOIN` en SQL, mais déclenché explicitement à la lecture plutôt qu’imposé par le schéma.

6. Indexation

`scripts/04-index.js` (repris de `scripts/indexes/create-indexes.js` , utilisé par l’application réelle) crée l’ensemble des index suivants :

Collection	Champ(s)	Type	Justification
<code>users</code>	<code>email</code>	Unique	Authentification — garantit l’unicité du compte
<code>users</code>	<code>addresses.coordinates</code>	Géospatial 2dsphere	Recherche d’utilisateurs/ adresses à proximité
<code>restaurants</code>	<code>address.coordinates</code>	Géospatial 2dsphere	Recherche de restaurants à proximité du client
<code>restaurants</code>	<code>address.district</code> , <code>isOpen</code>	Composé	Liste des restaurants ouverts d’un quartier (route la plus consultée du catalogue)
<code>restaurants</code>	<code>name</code> , <code>menus.name</code> , <code>menus.dishes.name</code> , <code>menus.dishes.description</code>	Texte pondéré	Recherche plein-texte (poids plus fort sur le nom du restaurant et du plat que sur la description)
<code>commandes</code>	<code>status</code> , <code>restaurant_id</code>	Composé	Tableau de bord vendeur — commandes actives d’un restaurant donné (index le plus critique du projet, voir §7)
<code>commandes</code>	<code>customer_id</code> , <code>metadata.createdAt</code> (desc)	Composé	Historique des commandes d’un client, trié du plus récent au plus ancien

Collection	Champ(s)	Type	Justification
commandes	deliverer_id, status	Composé	Suivi des livraisons en cours d'un livreur
commandes	deliveryTracking.coordinates	Géospatial 2dsphere	Requêtes de proximité sur le tracé GPS
deliverers	currentLocation	Géospatial 2dsphere	Recherche des livreurs disponibles à proximité d'un restaurant
deliverers	isAvailable, isActive	Composé	Filtrage des livreurs disponibles pour une nouvelle attribution

Le socle obligatoire exige au minimum trois index pertinents dont un composé : ce projet en compte onze, dont **cinq index composés**, largement au-delà du minimum, car le modèle multi-rôles (client/restauranteur/livreur) génère naturellement plusieurs angles de filtrage fréquents sur la collection commandes.

L'ordre des champs de chaque index composé suit la règle ESR (*Equality, Sort, Range*) : le champ filtré par égalité stricte (status, customer_id, deliverer_id) est placé en tête, ce qui permet à MongoDB de localiser directement la bonne plage de l'index avant d'appliquer un éventuel tri.

7. Analyse des performances

La preuve de performance porte sur l'index le plus critique du projet, commandes (status, restaurant_id) — celui qui alimente le tableau de bord du restaurateur, consulté à chaque rafraîchissement de page. La mesure complète (captures d'écran mongosh incluses) est disponible dans explain/explain-avant-apres.pdf et docs/design/PHASE5_EXPLAIN_CAPTURE.md. Méthode : l'index a été temporairement supprimé (dropIndex) pour mesurer le plan d'exécution sans index, puis recréé à l'identique pour mesurer le plan avec index, sur une collection commandes portée à **500 documents** (jeu de données de démonstration dédié, voir section 9) afin que l'écart de performance soit significatif.

Requête de référence (cas d'usage réel — lister les commandes CONFIRMED d'un restaurant) :

```
db.commandes.find({ status: "CONFIRMED", restaurant_id:
  ObjectId("6a2feaba46b300b4a537ec97") })
```

Métrique	Sans index (COLLSCAN)	Avec index (FETCH ← IXSCAN)
Stage gagnant	COLLSCAN	FETCH ← IXSCAN
Documents examinés (totalDocsExamined)	500 (collection entière)	12
	0	12

Métrique	Sans index (COLLSCAN)	Avec index (FETCH ← IXSCAN)
Clés d'index examinées (totalKeysExamined)		
Documents retournés (nReturned)	12	12
Ratio examinés / retournés	41,7	1

Interprétation. Sans index, MongoDB doit parcourir l'intégralité des 500 documents de la collection pour n'en retenir que 12 — le coût de la requête croît linéairement avec la taille totale de la collection, **indépendamment** de la sélectivité du filtre. Avec l'index composé, MongoDB localise directement les 12 documents pertinents via une recherche dans l'arbre B de l'index (IXSCAN), puis ne va chercher sur disque (FETCH) que ces 12 documents : le coût ne dépend plus que du nombre de résultats réellement retournés. Le ratio passe de 41,7 à 1.

Cet écart, déjà significatif sur 500 documents, se creuserait fortement à l'échelle de production : avec, par exemple, 50 000 commandes accumulées et la même sélectivité, un COLLSCAN examinerait 50 000 documents pour le même résultat de 12 lignes, tandis que l' IXSCAN resterait à un coût quasi constant — c'est précisément ce que le sujet demande de démontrer (« passage de COLLSCAN à IXSCAN »).

8. Comparaison SQL contre NoSQL

Le sujet impose une comparaison explicite entre l'approche relationnelle et l'approche documentaire choisie. Nous la construisons à partir d'un cas concret du projet : représenter une commande et son restaurant.

8.1 Le même besoin en modèle relationnel

Un schéma SQL équivalent imposerait, au minimum, les tables suivantes (clés étrangères en gras) :

```
CREATE TABLE restaurants (id, name, district, lat, lng, is_open, rating, owner_id
    FK);
CREATE TABLE dishes (id, restaurant_id FK, name, price, category,
    is_available, quantity);
CREATE TABLE delivery_zones (id, restaurant_id FK, zone, delivery_fee,
    delivery_time);
CREATE TABLE orders (id, customer_id FK, restaurant_id FK, deliverer_id FK,
    status, subtotal, delivery_fee, total, currency);
CREATE TABLE order_items (id, order_id FK, dish_id FK, dish_name, quantity,
    unit_price, total_price);
CREATE TABLE order_status_history (id, order_id FK, status, timestamp, note);
CREATE TABLE delivery_tracking (id, order_id FK, timestamp, lat, lng, speed);
```

Sept tables sont nécessaires pour représenter ce qu'une seule collection commandes enrichie de sous-documents (items[] , statusHistory[] , deliveryTracking[]) exprime en un document MongoDB unique.

8.2 Conséquences pratiques de chaque modèle

Critère	Relationnel (SQL)	Documentaire (MongoDB, ce projet)
Lecture d'une commande complète (articles + historique + tracé GPS)	4 JOIN minimum (order_items, order_status_history, delivery_tracking, + jointures vers customers / restaurants / deliverers pour les noms)	1 findOne, document déjà complet (les jointures vers restaurants / users restent nécessaires mais sont ponctuelles, via \$lookup ou en deux requêtes côté application)
Évolution du menu d'un restaurant sans casser l'historique	Nécessite une table order_items qui dénormalise déjà dish_name / unit_price au moment de l'achat — c'est-à-dire que même en SQL, on retrouve le même principe de « snapshot » que notre items[] embarqué	Snapshot natif via l'embedding, sans table de jointure dédiée
Schéma hétérogène (un menu de restaurant gabonais vs un menu international avec d'autres attributs)	Migration de schéma (ALTER TABLE) ou colonnes nullable en prévision de cas futurs	Chaque document restaurants porte son propre schéma de menus[], sans migration
Garantie d'intégrité référentielle stricte (un order_id orphelin est impossible)	Native (contraintes FOREIGN KEY)	Absente nativement — la cohérence entre commandes.restaurant_id et restaurants._id est garantie par la couche applicative (Mongoose + contrôleurs), pas par la base
Transactions multi-tables ACID par défaut	Natif, sans configuration particulière	Disponible (sessions/transactions MongoDB ≥ 4.0) mais à activer explicitement — non nécessaire dans ce projet car les écritures critiques (décrément de stock, changement de statut) restent mono-document
Montée en charge horizontale (sharding)	Possible mais coûteuse en réécriture applicative (la jointure devient inter-nœuds)	Pensée dès la conception : commandes shardable par restaurant_id sans changer le modèle applicatif
Requêtes analytiques ad hoc (jointures arbitraires, GROUP BY complexes)	Très mature (SQL déclaratif, optimiseur de requêtes ad hoc)	Possible via le pipeline d'agrégation (\$lookup, \$group), mais chaque jointure doit être explicitement programmée — moins flexible pour une requête imprévue

8.3 Pourquoi MongoDB pour ce cas d'usage

Le choix du documentaire se justifie ici par trois facteurs dominants : (1) le patron d'accès dominant de l'application est *lire une commande complète d'un seul coup* (panier client, suivi de livraison, tableau de bord vendeur), ce que l'embedding satisfait nativement sans jointure ; (2) le menu d'un restaurant est intrinsèquement semi-structuré et change de forme selon l'établissement, ce qui correspond au schéma flexible documentaire plutôt qu'à des colonnes fixes ; (3) le volume d'écritures fréquentes et de faible latence (positions GPS, statuts) justifie un couplage avec Redis, naturel dans l'écosystème NoSQL. À l'inverse, si le besoin dominant avait été des rapports financiers transverses avec des jointures arbitraires et une exigence d'intégrité référentielle stricte entre de nombreuses entités (cas typique d'un système de facturation comptable), un SGBD relationnel aurait été plus adapté — ce projet illustre un cas où le NoSQL documentaire est le bon choix, pas une règle universelle.

9. Difficultés rencontrées

Rendre la démonstration `explain()` significative. Le seed « officiel » de l'application (30 restaurants, environ 30 commandes pour rester réaliste) ne suffisait pas à rendre visible l'écart entre `COLLSCAN` et `IXSCAN` — sur un trop petit volume, MongoDB peut choisir un plan de table complète sans pénalité mesurable. Il a fallu écrire un script de seed complémentaire dédié (`scripts/demo/seed-commandes-demo.js`) portant la collection `commandes` à 500 documents pour la seule durée de la démonstration, sans polluer le jeu de données « réaliste » utilisé par ailleurs pour les captures d'écran applicatives.

Arbitrer l'embedding du tracé GPS. `deliveryTracking[]` est le sous-document dont la croissance est la moins prévisible du modèle (un point toutes les 20-30 secondes pendant toute la durée d'une livraison). Nous avons dû explicitement vérifier que cette croissance restait largement compatible avec la limite de 16 Mo par document avant de valider l'embedding (une livraison de plusieurs heures, hypothèse non réaliste pour de la restauration, resterait de plusieurs ordres de grandeur sous cette limite) ; la note de conception Redis (section 4.3) documente la stratégie de repli si ce choix devait être révisé à plus grande échelle (Redis Streams en tampon haute fréquence, persistance MongoDB allégée en fréquence).

Garder synchronisés deux niveaux de scripts. Le cahier des charges impose des scripts `mongosh` purs et autonomes (`scripts/01-crud.js` à `04-index.js`), tandis que l'application réelle utilise Mongoose (`scripts/seed`, `scripts/aggregations`, `scripts/indexes`). Maintenir une correspondance exacte entre les deux (mêmes pipelines, mêmes index, mêmes noms de champs) a demandé une vigilance particulière pour éviter que les scripts de démonstration ne dérivent du comportement réel de l'API — la solution adoptée a été de toujours écrire la version Mongoose en premier, puis de la transposer en `mongosh` pur, et de documenter explicitement cette filiation en commentaire de chaque script.

Éviter la double réservation de stock. Le défi technique du sujet implicite à la gestion de menu (décrément de `quantity` à chaque commande) pose un risque de concurrence classique : deux clients commandant simultanément le dernier exemplaire d'un plat. La modélisation retient ce champ au niveau du plat embarqué précisément pour permettre une mise à jour atomique

conditionnelle (`updateOne` avec un filtre `quantity: { $gt: 0 }` combiné à un `$inc: { quantity: -1 }`), ce qu'une mise à jour en deux temps (lecture puis écriture séparées) ne garantirait pas.

Répartition du travail en équipe de trois sur deux semaines. Construire en parallèle le socle obligatoire (scripts `mongosh`, conception, indexation) et l'ensemble des livrables bonus (API REST complète, frontend de démonstration) dans le délai imparti a nécessité de prioriser strictement le socle obligatoire avant tout travail sur les bonus, conformément à la recommandation du cahier des charges — l'API REST a ainsi été construite en réutilisant directement les schémas et la logique métier validés lors de la phase de conception, plutôt qu'en parallèle.

10. Conclusion

Ce projet démontre, sur un cas d'usage réel de livraison de repas à Libreville, l'ensemble des compétences NoSQL attendues par le module : une modélisation documentaire dont chaque choix embedding/referencing est justifié au regard des patrons d'accès réels de l'application (et non par défaut) ; les opérations CRUD complètes, y compris les opérateurs de mise à jour ciblée (`$set`, `$inc`, `$push`, `$pull`) ; des requêtes avancées combinant filtres, expressions régulières, tri, pagination et comptage ; des pipelines d'agrégation multi-étapes avec jointures (`$lookup`) reproduisant des besoins analytiques réels (chiffre d'affaires par restaurant, performance des livreurs) ; et une stratégie d'indexation mesurée par `explain('executionStats')`, avec une démonstration chiffrée du passage de `COLLSCAN` à `IXSCAN` sur l'index le plus critique du projet.

Au-delà du socle obligatoire, l'implémentation complète (API Express/Mongoose sécurisée par JWT, cache Redis avec stratégie de repli documentée, suivi temps réel par Socket.io, frontend Next.js de démonstration) montre comment ces choix de modélisation se traduisent en conditions réelles : la machine à états des commandes (`src/utils/orderStateMachine.js`) garantit l'intégrité du cycle de vie d'une commande sans contrainte de base de données ; l'invalidation de cache (`src/services/cacheService.js`) reste cohérente avec les clés MongoDB ; et les index géospatiaux sont effectivement exploités par les fonctionnalités de proximité (restaurants et livreurs à proximité) plutôt que de rester un exercice isolé.

Pistes d'amélioration

- **Sharding** de `commandes` par `restaurant_id` si le volume dépassait l'échelle d'un seul nœud, conformément à la réflexion menée en section 8.2.
- **Time-series collection MongoDB native** (introduite en MongoDB 5.0) pour `deliveryTracking`, qui formaliserait la vigilance documentée en section 9 sur la croissance de ce sous-document, au-delà d'un certain volume de points GPS par commande.
- **Vues matérialisées** (collection dédiée, recalculée par triggers) pour les statistiques agrégées les plus consultées (tableau de bord vendeur), afin d'éviter de recalculer les pipelines d'agrégation à chaque requête.
- **Transactions MongoDB explicites** pour les écritures touchant simultanément plusieurs documents (par exemple, décrémentation de stock et création de commande), si ce besoin venait à dépasser le cadre des mises à jour atomiques mono-document actuellement suffisantes.